

Prueba tecnica - Plataforma de gestion logistica

Reto de 3 dias - Backend, frontend y despliegue cloud equilibrados

Duracion

El candidato cuenta con 3 dias calendario para entregar una solucion funcional, documentada y defendible en entrevista tecnica.

Contexto

Una empresa de logistica esta modernizando parte de su operacion. Actualmente existen procesos antiguos para registrar clientes, paquetes, cambios de estado y novedades. Se requiere construir una solucion nueva que permita operar el flujo principal, consultar trazabilidad y proponer una forma segura de llevarla a produccion.

La prueba busca evaluar criterio tecnico integral: construccion de backend, experiencia de usuario en frontend y decisiones de infraestructura/operacion. No se espera una plataforma completa, sino una solucion pequena, coherente y explicable.

Objetivo general

- Construir una aplicacion funcional para registrar clientes y paquetes.
- Permitir cambio de estados con reglas, auditoria e historial.
- Exponer una interfaz web usable para operar y consultar la informacion.
- Procesar acciones derivadas sin bloquear la operacion principal.
- Proponer una arquitectura productiva segura, observable y recuperable.

Alcance equilibrado esperado

Frente	Peso sugerido	Resultado esperado
Backend	33%	API funcional, modelo de datos, reglas de negocio, seguridad, procesos derivados y pruebas basicas.
Frontend	33%	Interfaz web para login o acceso, listado filtrable, registro/edicion basica, detalle de paquete, trazabilidad y manejo de errores.
Cloud / operacion	34%	Propuesta productiva con red, seguridad, despliegue, secretos, logs, monitoreo, backups, escalabilidad y recuperacion ante fallos.

Regla importante sobre tecnologia

El candidato tiene libertad para elegir las herramientas que considere adecuadas. En el README debe justificar por que eligio su stack y que alternativas descarto. Se evaluara especialmente si la seleccion tecnica es coherente con el problema, no si solo cumple una lista de moda, esa coleccion triste de logos en repositorios.

Requerimientos funcionales

1. Gestion de usuarios y acceso

- Debe existir un mecanismo basico de acceso a la aplicacion.
- Debe manejar al menos tres perfiles: administrador, operador y consulta.
- Administrador: puede gestionar clientes y usuarios.
- Operador: puede crear paquetes y cambiar estados.
- Consulta: solo puede ver informacion.
- La interfaz debe ocultar o bloquear acciones que el usuario no tenga permiso de ejecutar.

2. Gestion de clientes

- Registrar clientes con nombre, documento/NIT, correo, telefono y estado activo/inactivo.
- Consultar clientes registrados.
- Validar que no se creen clientes incompletos.
- Impedir operaciones criticas sobre clientes inactivos, segun criterio del candidato.

3. Gestion de paquetes

- Registrar paquetes asociados a un cliente existente.
- Datos minimos: codigo unico, destinatario, direccion, ciudad, departamento, telefono, peso, valor declarado y fecha de creacion.
- El codigo del paquete no se puede repetir.
- El estado inicial debe ser asignado automaticamente por el sistema.
- La interfaz debe permitir crear paquete y consultar el resultado sin recargar manualmente toda la aplicacion.

4. Estados y reglas de transicion

Estados minimos: Registrado, Recogido, En centro de distribucion, En reparto, Entregado, Novedad, Devuelto y Cancelado.

- No se debe permitir cualquier cambio de estado.
- Un paquete entregado no puede volver a estar en reparto.
- Un paquete cancelado no puede cambiar a otro estado.
- Un paquete devuelto no puede marcarse como entregado directamente.
- Para marcar como entregado debe existir nombre de quien recibe.
- Para marcar como novedad debe existir observacion.
- El candidato puede agregar reglas adicionales si las justifica.

5. Historial y trazabilidad

- Cada cambio de estado debe guardar estado anterior, estado nuevo, fecha, usuario/responsable, observacion y datos adicionales cuando aplique.
- La trazabilidad debe ser consultable desde la API y desde la interfaz web.
- El historial debe ordenarse cronologicamente y no debe perderse cuando cambia el estado actual.
- La pantalla de detalle debe mostrar datos del paquete, estado actual, historial y acciones derivadas.

6. Procesos derivados

Algunos cambios de estado deben generar acciones adicionales. Estas acciones no deben bloquear la respuesta principal al usuario.

- Al marcar Entregado: generar registro de liquidacion.
- Al marcar Novedad: generar alerta operativa.
- Al marcar Devuelto: generar accion pendiente de revision.
- El sistema debe registrar si la accion derivada esta pendiente, procesada o fallida.
- Debe existir una forma de consultar estas acciones y entender que fallo si no se procesan.

7. Consulta operativa

- Listado de paquetes con filtros por cliente, estado, ciudad, rango de fechas y código.
- Paginación obligatoria.
- La interfaz debe mostrar estados de carga, vacío, error y resultado.
- La experiencia debe ser usable en pantalla de escritorio. Responsividad básica suma puntos.

8. Dashboard mínimo

- Mostrar un resumen con conteo de paquetes por estado.
- Mostrar últimos cambios de estado.
- Mostrar acciones derivadas pendientes o fallidas.
- No se requiere analítica avanzada; se busca criterio para exponer información operativa útil.

9. Seguridad

- Validación de entradas en backend y frontend.
- Control de permisos en backend, no solo ocultar botones en frontend.
- Manejo seguro de credenciales y variables de entorno.
- Errores controlados, sin exponer información sensible.
- Prevención de consultas inseguras.
- No subir secretos reales al repositorio.

10. Propuesta productiva cloud

Debe incluir una propuesta de despliegue productivo. No es obligatorio desplegar la prueba en la nube, pero sí explicar claramente cómo se haría.

- Separación de componentes: frontend, backend, base de datos y procesos en segundo plano.
- Red y exposición: qué queda público, qué queda privado y por qué.
- Seguridad de acceso administrativo.
- Manejo de secretos.
- Logs, monitoreo y alertas.
- Backups y recuperación.
- Escalabilidad inicial y puntos de falla.
- Estrategia básica de CI/CD o despliegue controlado.

Entregables

- Repositorio con código fuente completo.
- Backend ejecutable localmente.
- Frontend ejecutable localmente.
- Persistencia de datos real o claramente simulada con justificación técnica. Para una prueba de 3 días se valora persistencia real.
- Archivo README con instalación, ejecución, variables de entorno, pruebas y decisiones.
- Archivo DECISIONS.md con trade-offs, arquitectura elegida, alternativas descartadas y mejoras futuras.
- Modelo de datos: diagrama, migraciones o scripts.
- Diagrama de arquitectura productiva.
- Colección de peticiones, documentación API o Swagger/OpenAPI.
- Evidencia de funcionamiento: capturas, video corto o pasos reproducibles.

Requisitos mínimos de calidad

- La solución debe poder ejecutarse con instrucciones claras.
- No se aceptan respuestas solo teóricas.
- No se aceptan soluciones donde el frontend sea una página decorativa sin flujo real.
- No se aceptan soluciones donde el backend no persista cambios de estado.
- No se aceptan soluciones sin explicación de seguridad y despliegue.

- No se aceptan secretos reales ni archivos de entorno con credenciales productivas.

Criterios de evaluacion

Categoria	Puntos	Que se revisa
Backend y reglas de negocio	20	API, capas, validaciones, estados, historial, errores, autorizacion y transacciones.
Procesos derivados y asincronia	13	Ejecucion no bloqueante, reintentos, idempotencia, recuperacion de fallos y consulta de estado.
Frontend y experiencia de usuario	25	Flujos completos, formularios, filtros, detalle, trazabilidad, manejo de permisos, errores y estados de carga.
Base de datos	10	Modelo, relaciones, restricciones, indices, consultas paginadas e integridad.
Cloud / operacion / seguridad productiva	22	Red, componentes, seguridad, secretos, logs, monitoreo, backups, escalabilidad, CI/CD y recuperacion.
Documentacion, pruebas y defensa	10	README, DECISIONS, pruebas, evidencia, claridad y capacidad de explicar decisiones.

Defensa tecnica posterior

- Explicar arquitectura general y decisiones de stack.
- Explicar flujo frontend-backend-base de datos al crear y cambiar estado de un paquete.
- Explicar como se procesan acciones derivadas sin duplicarlas.
- Explicar como se desplegaria la solucion en produccion de forma segura.
- Realizar un cambio pequeno en vivo sobre su propia solucion.
- Responder preguntas de rendimiento, seguridad, escalabilidad y mantenibilidad.

Cambios que se podran pedir en vivo

- Agregar un nuevo filtro al listado frontend y conectarlo al backend.
- Agregar una regla nueva de transicion.
- Agregar un nuevo rol o permiso.
- Hacer que una accion derivada no se duplique si se procesa dos veces.
- Corregir una validacion visible en frontend y backend.
- Explicar que pasa si el worker/proceso en segundo plano falla.
- Explicar que cambia para desplegar backend y frontend en ambientes separados.